

Sorting I : Count Sort & Merge Sort

Hello everyone :)

Nipun Mittal **googler**

BTech - 2019, MTech - 2021 (IIT Bombay)

~ 2 years of part-time teaching experience

Ques! find the smallest no. that can be formed by rearranging the digits of a given no. in array.

$$0 \leq A[i] \leq 9$$

$$A = [3, 2, 4, 1] \rightarrow [1, 2, 3, 4]$$

$$A = [6, 3, 4, 2, 7, 2, 0] \rightarrow [0, 2, 2, 3, 4, 6, 7]$$

Solⁿ: sorting in ascending order

Inbuilt sorting: $TC = O(N \log N)$

output will always look like:

0000... 111... 22 ... 33 88 ... 99 ...
freq(0) freq(1) ...

$$A = \begin{bmatrix} 9 & 8 & 9 & 7 & 1 & 1 & 9 & 0 & 0 & 2 & 9 & 1 & 3 & 7 \end{bmatrix}$$
$$f = \begin{bmatrix} \cancel{0}^{12} & \cancel{0}^{12^3} & \cancel{0}^1 & \cancel{0}^1 & 0 & 0 & 0 & \cancel{0}^{12} & \cancel{0}^1 & \cancel{0}^{+2^3^4} \end{bmatrix}$$

$f(i) \rightarrow$ freq of i (0-9) in the array

$$f = \begin{bmatrix} 2 & 3 & 1 & 1 & 0 & 0 & 0 & 2 & 1 & 4 \\ 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \end{bmatrix}$$

→ 0 0 1 1 1 2 3 7 7 8 9 9 9 9

$$\forall i, f(u) = 0 \quad // \text{len} = 10$$

Count Sort

```
for (i = 0 to n-1) {  
    f[A[i]]++  
}
```

} calculating frequency

N iterations

```
for ( d = 0 to 9 ) {  
    for ( i = 1 to f[d] ) {  
        print(d)  
    }  
}
```

sort array
using
frequency

total

iterations $\rightarrow 10 \times N$



N ✓

total TC : $O(N)$

$$SC = O(10) = O(1)$$

d	i	iteration
0	1...10	f(0)
1		f(1)
⋮		⋮
9		f(9)

$$\Rightarrow f(0) + f(1) + \dots + f(9) = N$$

What if $0 \leq A[i] \leq 10^9 \rightarrow$ Is CountSort possible?

length of freq array = 10^9
 ↓
 int array
 4B

$$4B \times 10^9 \times 10^3 \times 10^3$$

4KB

4MB

~ 4GB (Array space)

↓
 Memory Limit Exceeded
 error
 (MLE)

$[10^6 - 10^9]$

What if $-9 \leq A[i] \leq 9 \rightarrow$ Is CountSort possible?

$$\text{len} = [-9, 9] \Rightarrow 9 - (-9) + 1 = 19 \quad \underline{\underline{\text{yes}}}$$

$$A = \begin{bmatrix} -2 & 3 & 8 & -4 & -2 & 3 & 0 \end{bmatrix}$$

$$F = \begin{bmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 & 15 & 16 & 17 & 18 \end{bmatrix}$$

$$\begin{bmatrix} -9 & -8 & -7 & \dots & -3 & -2 & -1 & 0 & 1 & 2 & 3 & \dots & 8 & 9 \end{bmatrix}$$

$$\forall i, f(i) = 0 \quad // \text{len} = 19$$

$$\text{for } (i = 0 \text{ to } n-1) \{$$

$$\text{f[A[i]]} \rightarrow \text{f[A[i] + 9]} \quad // \text{f[A[i]] = minElement}$$

in this case minElement = -9

}

$$\text{for } (d = 0 \text{ to } \text{len}-1) \{$$

$$\text{for } (i = 1 \text{ to } f[d]) \{$$

$$\text{print(d)} \rightarrow \text{print(d - 9)} \quad // \text{print(d + minElement)}$$

}

}

$$TC = O(N)$$

$$SC = O(\text{range of array})$$

$$(\max(A[i]) - \min(A[i]) + 1) \leq 10^6$$

use Count Sort

$$[-10^3, 10^3]$$

$$\Rightarrow 10^3 - (-10^3) + 1$$

$$\Rightarrow 2 \times 10^3 + 1 < 10^6 \quad \checkmark$$

Ques Given an integer array where all odd elements are sorted & all even elements are sorted, sort the array.

$A = [\overset{0}{2} \overset{1}{5} \overset{2}{4} \overset{3}{8} \overset{4}{11} \overset{5}{13} \overset{6}{10} \overset{7}{15} \overset{8}{21}]$

$[\overset{0}{2} \overset{1}{4} \overset{2}{8} \overset{3}{10}]$
 $i \quad i \quad i \quad i \quad i$

$[\overset{0}{5} \overset{1}{11} \overset{2}{13} \overset{3}{15} \overset{4}{21}]$
 $j \quad j$

$TC = O(N)$
 $SC = O(N)$

$TC = O(N)$

$[2 \quad 4 \quad 5 \quad 8 \quad 10 \quad 11 \quad 13 \quad 15 \quad 21]$

Merge 2 sorted array of size N & M

$$TC = O(N+M)$$

$$SC = O(N+M)$$

```
int merge ( A[], N, B[], M ) {
```

```
    ans [ N+M]
```

```
    i = 0 , j = 0 , k = 0
```

```
    while ( i < N && j < M ) {
```

```
        if ( A[i] <= B[j] ) {
```

```
            ans[k] = A[i]
```

```
            i++
```

```
        }
```

```
        else {
```

```
            ans[k] = B[j]
```

```
            j++
```

```
        }
```

```
        k++
```

```
    }
```

```
    while ( i < N ) {
```

```
        ans[k] = A[i]
```

```
        k++, i++
```

```
    }
```

```
    while ( j < M ) {
```

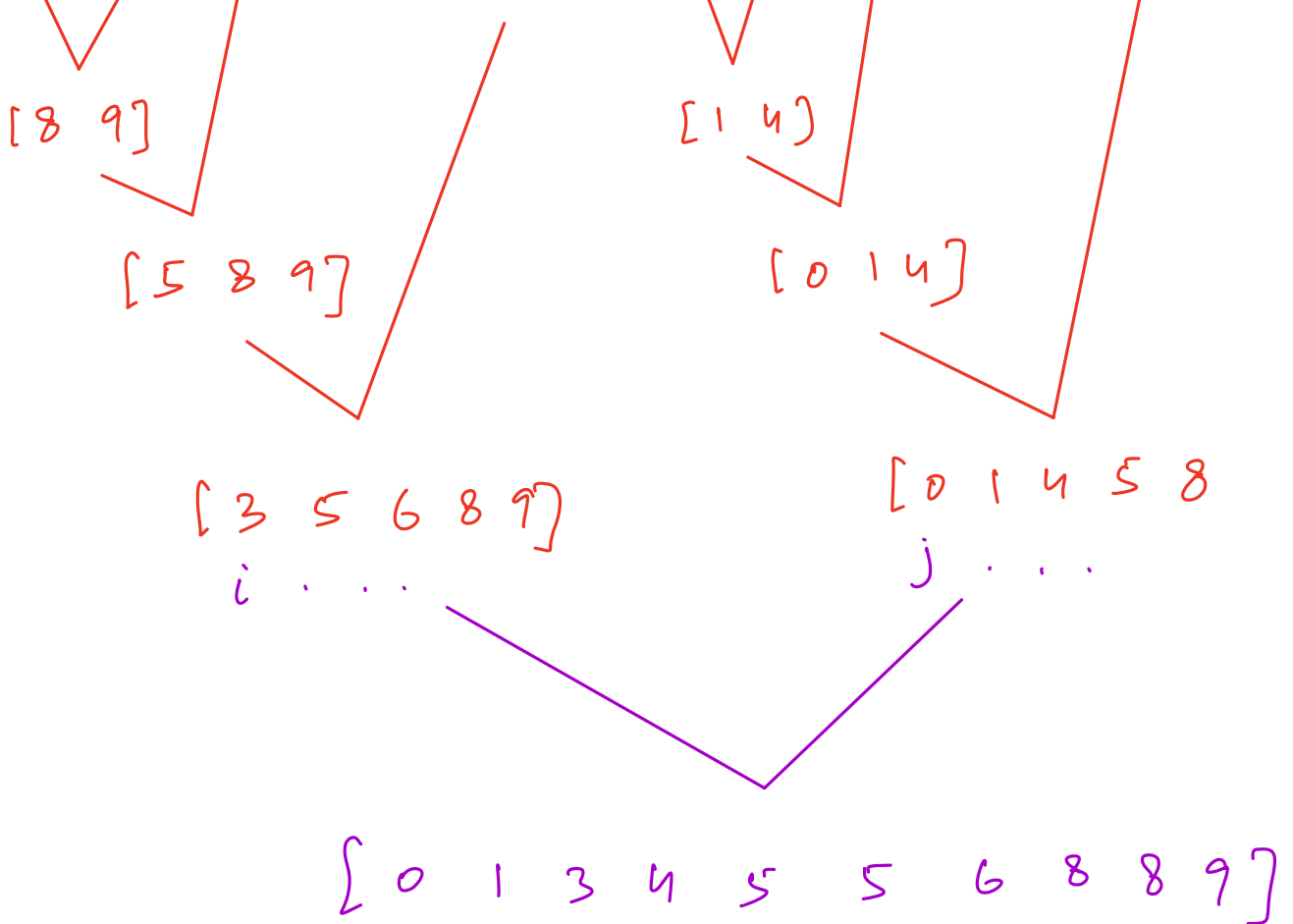
```
        ans[k] = B[j]
```

```
        k++, j++
```

```
    }
```

$TC = O(N+M)$

$SL = O(N+M)$



```
void mergeSort ( A[], 0l, N-1r) {
```

```
    if ( l >= r ) return;
```

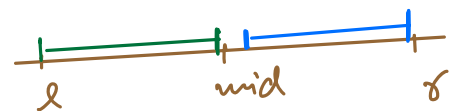
```
    mid = (l+r)/2
```

```
    mergeSort ( A, l, mid)
```

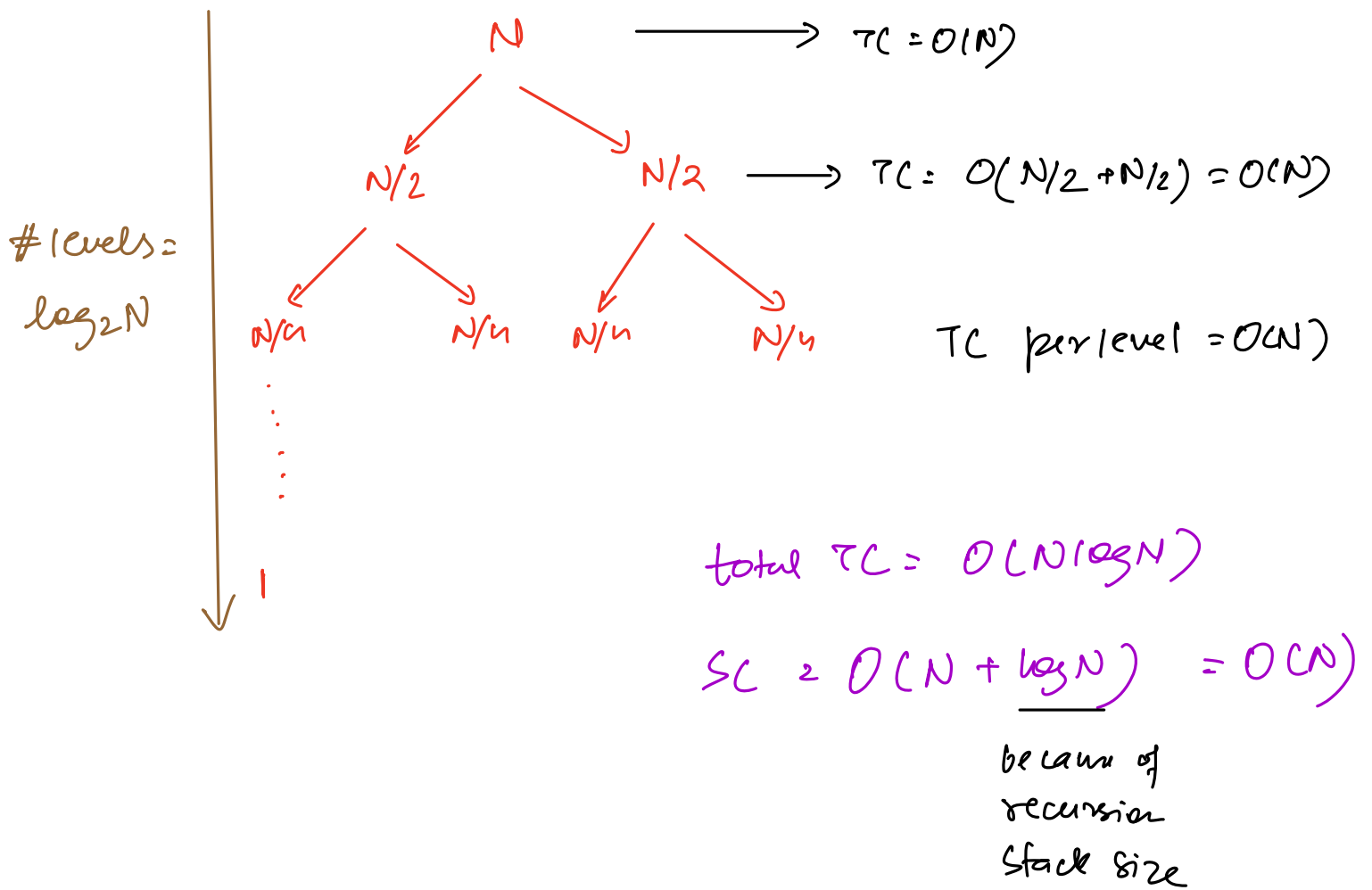
```
    mergeSort ( A, mid+1, r)
```

```
    merge ( A, l, mid, r)
```

$\rightarrow T.C = O(N \log N), S.C = O(N)$



}



Question

Given an array, find count of inversion pairs.

inversion pair $(i, j) \Rightarrow i < j \text{ \& \& } A[i] > A[j]$

$$A = [\overset{0}{10} \quad \overset{1}{3} \quad \overset{2}{8} \quad \overset{3}{15} \quad \overset{4}{6}]$$

$i < j$		$A[i] > A[j]$	
0	1	$10 > 3$	✓
0	2	$10 > 8$	✓
0	3	$10 > 15$	✗
0	4	$10 > 6$	✓
1	2	$3 > 8$	✗
1	3	$3 > 15$	✗
1	4	$3 > 6$	✗
2	3	$8 > 15$	✗
2	4	$8 > 6$	✓
3	4	$15 > 6$	✓

ans = 5

Brute force $\Rightarrow$ TC = $O(N^2)$

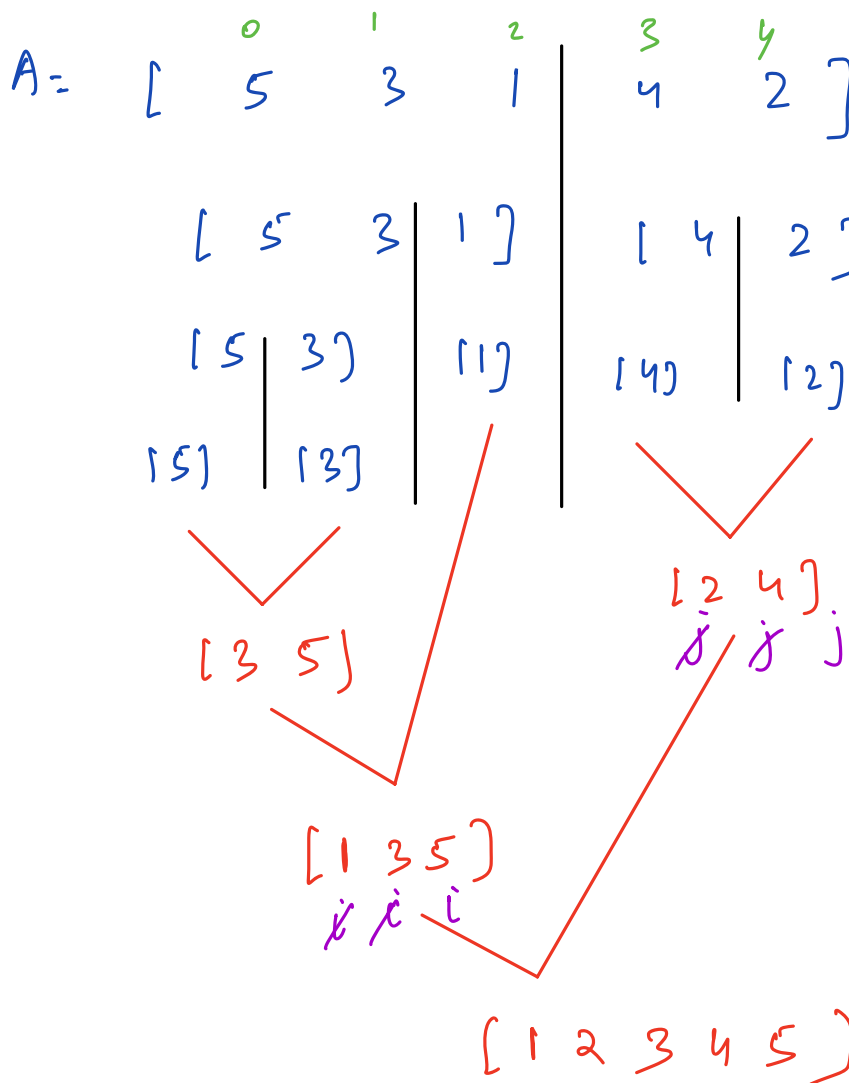
SC = $O(1)$

$A = [\overset{0}{5} \overset{1}{3} \overset{2}{1} \overset{3}{4} \overset{4}{2}]$

$$ans = 0 + 1 + 2 + 1 + 3 = 7$$

$i < j \quad \&\& \quad A[i] > A[j]$

count # of greater elements in left



$$\begin{aligned}
 ans &= f1 \\
 &+ 2 \\
 &+ 1 \\
 &+ 2 \\
 &+ 1 \\
 \hline
 &= 7
 \end{aligned}$$

if selecting right side element while merging

$ans += \# \text{ remaining elements in left}$

$$TC = O(N \log N)$$

$$SC = O(N)$$

$\begin{matrix} 0 & 1 & 2 & 3 \\ [& 5 & 2 & 6 & 1 &] \\ \downarrow & \downarrow & \downarrow & \downarrow \\ 0 & 1 & 0 & 3 \end{matrix}$
 $\Rightarrow 1+3=4$

$(0,1)$ $(0,3)$
 $(1,3)$
 $(2,3)$

$\begin{matrix} 0 & 1 & 2 & 3 & 4 \\ [& 5 & 3 & 1 & 4 & 2 &] \end{matrix}$
 $ans = 7$

$(0,1)$ $(3,4)$ $(0,4)$
 $(0,2)$ $(1,4)$ $(0,3)$
 $(1,2)$

int merge (C[], l, mid, r) {

$[l, mid]$
 $[mid+1, r]$

$N = mid - l + 1$, $M = r - (mid + 1) + 1 = r - mid$

$A[0 \dots N-1] = C[l \dots mid]$
 $B[0 \dots M-1] = C[mid+1 \dots r]$

create A[] & B[]

$i = 0$, $j = 0$, $k = 0$

while (i < N & j < M) {

if (A[i] < B[j]) {

preserves
stable order

C[k] = A[i]

i++

}

else { // inversion

```

        C[k] = B[j]
        j++
        count += N - i
    }
    k++
}

while (i < N) {
    C[k] = A[i]
    k++, i++
}

while (j < M) {
    C[k] = B[j]
    k++, j++
}

```

Stable sorting → Relative order of equal elements should not change while sorting w.r.t a parameter.

$A = [6, 5, 3, 5]$

↳ $[3, 5, 5, 6]$

Name Marks

A	8
B	5
C	8
D	4
E	8

sort
→
w.r.t marks

D	4
B	5
A	8
C	8
E	8

Inplace sorting

if no extra space is needed to sort, it is called
inplace sorting.

if $SC = O(1) \Rightarrow$ inplace